

Google Interview Preparation — Database & SQL Roles

1. Company Profile

Engineering Culture

- **Data-driven everything:** every product decision backed by BigQuery analytics
- Peer review culture: code/design reviews by multiple engineers before ship
- "20% time" innovation mindset — side projects that become products
- Strong emphasis on scalability: designs must handle 10x-1000x growth
- Academic rigor: Google values computer science fundamentals deeply
- Distributed systems at planetary scale (Spanner, BigTable, Bigtable, F1, Dremel)

Database Technology Stack

Layer	Technology
Global OLTP	Cloud Spanner (globally distributed SQL)
Analytics	BigQuery (serverless, petabyte-scale)
Key-Value	Bigtable (HBase-compatible)
Caching	Memcached, Redis
PostgreSQL	Cloud SQL for PostgreSQL (customer-facing products)
Search	Custom inverted index, Elasticsearch
Time Series	Monarch (internal), Cloud Monitoring
Data Lake	GCS + Dataflow + BigQuery
Graph	Graph databases for Knowledge Graph

What Google Values in DB/SQL Candidates

- Deep algorithmic thinking applied to query optimization
 - Understanding of distributed systems (Paxos, 2PC, consensus)
 - Clean, maintainable SQL — not just "it works"
 - Awareness of execution plans and cost models
 - Ability to discuss trade-offs rigorously with data
 - Scale-first thinking from day one
-

2. Interview Process

Typical Rounds (L4/L5/L6 Software Engineer, Data Engineer, SRE)

Round	Format	Duration	Focus
Recruiter Screen	Phone	30 min	Background, interest, expectations

Technical Phone 1	CoderPad	45–60 min	SQL + general coding
Technical Phone 2	CoderPad	45–60 min	System design or SQL
Onsite Round 1	Video/In-person	45 min	SQL + algorithms
Onsite Round 2	Video/In-person	45 min	System design with DB
Onsite Round 3	Video/In-person	45 min	Distributed DB / internals
Onsite Round 4	Video/In-person	45 min	Behavioral / Googleyness
Onsite Round 5	Video/In-person	45 min	Additional coding or design

Level Expectations

Level	Equivalent	SQL Expectation	System Design
L3	Junior	Basic to medium SQL	Not required
L4	Mid-level	Medium to hard SQL	High-level
L5	Senior	Hard SQL + optimization	Full design + trade-offs
L6	Staff	Expert + novel solutions	Architecture-level

3. SQL Coding Round — 25 Query Optimization Problems

Core Query Problems

Q1. Find all pairs of users who share the same city.

```
-- Naive: O(n2) cross join – BAD
-- Better:
SELECT a.user_id, b.user_id, a.city
FROM users a
JOIN users b ON a.city = b.city AND a.user_id < b.user_id;

-- With index: CREATE INDEX ON users (city, user_id);
```

Q2. Calculate percentile distribution of response times.

```
SELECT
  PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY response_ms) AS p50,
  PERCENTILE_CONT(0.95) WITHIN GROUP (ORDER BY response_ms) AS p95,
  PERCENTILE_CONT(0.99) WITHIN GROUP (ORDER BY response_ms) AS p99,
  PERCENTILE_CONT(1.0) WITHIN GROUP (ORDER BY response_ms) AS p100
FROM api_logs
WHERE logged_at >= NOW() - INTERVAL '1 hour';
```

Q3. Find the Nth highest value without using LIMIT.

```
-- Using DENSE_RANK (handles ties correctly)
SELECT DISTINCT value
FROM (
    SELECT value, DENSE_RANK() OVER (ORDER BY value DESC) AS rnk
    FROM scores
) ranked
WHERE rnk = :N;
```

Q4. Detect duplicate rows and keep only the latest.

```
-- Find duplicates
SELECT email, COUNT(*) FROM users GROUP BY email HAVING COUNT(*) > 1;

-- Delete keeping latest
DELETE FROM users
WHERE user_id NOT IN (
    SELECT DISTINCT ON (email) user_id
    FROM users
    ORDER BY email, created_at DESC
);
```

Q5. Find users who completed a funnel: viewed → added to cart → purchased.

```
WITH funnel AS (
    SELECT
        user_id,
        MAX(CASE WHEN event_type = 'view' THEN 1 END) AS viewed,
        MAX(CASE WHEN event_type = 'cart' THEN 1 END) AS carted,
        MAX(CASE WHEN event_type = 'purchase' THEN 1 END) AS purchased
    FROM events
    GROUP BY user_id
)
SELECT
    COUNT(*) FILTER (WHERE viewed = 1) AS total_views,
    COUNT(*) FILTER (WHERE carted = 1) AS total_carts,
    COUNT(*) FILTER (WHERE purchased = 1) AS total_purchases,
    ROUND(100.0 * COUNT(*) FILTER (WHERE carted = 1) / NULLIF(COUNT(*) FILTER (WHERE
viewed=1),0), 2) AS view_to_cart,
    ROUND(100.0 * COUNT(*) FILTER (WHERE purchased = 1) / NULLIF(COUNT(*) FILTER
(WHERE carted=1),0), 2) AS cart_to_purchase
FROM funnel;
```

Q6. Google-style: Find users whose search queries resulted in zero clicks (high impression, zero CTR).

```
SELECT
    query_text,
    SUM(impressions) AS total_impressions,
    SUM(clicks) AS total_clicks,
    ROUND(100.0 * SUM(clicks) / NULLIF(SUM(impressions), 0), 4) AS ctr_pct
```

```

FROM search_logs
GROUP BY query_text
HAVING SUM(clicks) = 0 AND SUM(impressions) > 1000
ORDER BY total_impressions DESC;

```

Q7. Compute the graph shortest path using recursive CTE.

```

WITH RECURSIVE paths AS (
    SELECT source_node, dest_node, 1 AS hops, ARRAY[source_node, dest_node] AS path
    FROM edges
    WHERE source_node = :start_node

    UNION ALL

    SELECT p.source_node, e.dest_node, p.hops + 1, p.path || e.dest_node
    FROM paths p
    JOIN edges e ON p.dest_node = e.source_node
    WHERE e.dest_node <> ALL(p.path) -- avoid cycles
        AND p.hops < 10 -- max depth guard
)
SELECT * FROM paths WHERE dest_node = :end_node
ORDER BY hops
LIMIT 1;

```

Q8. Calculate the Jaccard similarity between two sets (used in recommendation engines).

```

-- Items liked by user A and user B
WITH user_a_items AS (
    SELECT product_id FROM ratings WHERE user_id = :user_a AND rating >= 4
),
user_b_items AS (
    SELECT product_id FROM ratings WHERE user_id = :user_b AND rating >= 4
),
intersection AS (
    SELECT COUNT(*) AS common FROM user_a_items INTERSECT SELECT product_id FROM
user_b_items
),
union_set AS (
    SELECT COUNT(DISTINCT product_id) AS total
    FROM (SELECT product_id FROM user_a_items UNION SELECT product_id FROM
user_b_items) u
)
SELECT
    (SELECT common FROM intersection)::numeric /
    NULLIF((SELECT total FROM union_set), 0) AS jaccard_similarity;

```

Q9. Window frame: calculate exponential moving average (EMA).

```

-- PostgreSQL doesn't have EMA natively; implement with recursive CTE
WITH RECURSIVE ema AS (

```

```

SELECT
    date_col,
    price,
    price::numeric AS ema_value,
    ROW_NUMBER() OVER (ORDER BY date_col) AS rn
FROM stock_prices
WHERE symbol = 'GOOG'
LIMIT 1

UNION ALL

SELECT
    s.date_col,
    s.price,
    (s.price * 0.1 + ema.ema_value * 0.9)::numeric, -- alpha=0.1
    ema.rn + 1
FROM stock_prices s
JOIN ema ON s.symbol = 'GOOG'
        AND s.date_col = (SELECT MIN(date_col) FROM stock_prices WHERE date_col
> ema.date_col AND symbol = 'GOOG')
)
SELECT date_col, price, ROUND(ema_value, 4) AS ema FROM ema ORDER BY date_col;

```

Q10. Google Analytics: Calculate page view velocity — pages with fastest growing views in last 7 days vs. prior 7 days.

```

WITH recent AS (
    SELECT page_path, COUNT(*) AS recent_views
    FROM pageviews
    WHERE viewed_at >= NOW() - INTERVAL '7 days'
    GROUP BY page_path
),
prior AS (
    SELECT page_path, COUNT(*) AS prior_views
    FROM pageviews
    WHERE viewed_at BETWEEN NOW() - INTERVAL '14 days' AND NOW() - INTERVAL '7 days'
    GROUP BY page_path
)
SELECT
    r.page_path,
    r.recent_views,
    COALESCE(p.prior_views, 0) AS prior_views,
    ROUND(100.0 * (r.recent_views - COALESCE(p.prior_views, 0))
        / NULLIF(COALESCE(p.prior_views, 1), 0), 2) AS growth_pct
FROM recent r
LEFT JOIN prior p USING (page_path)
ORDER BY growth_pct DESC
LIMIT 20;

```

Query Optimization Problems

Q11. Rewrite a correlated subquery as a window function.

```
-- Slow: correlated subquery
SELECT e.employee_id, e.salary,
       (SELECT AVG(salary) FROM employees e2 WHERE e2.department_id = e.department_id)
AS dept_avg
FROM employees e;

-- Fast: window function (single pass)
SELECT employee_id, salary,
       AVG(salary) OVER (PARTITION BY department_id) AS dept_avg
FROM employees;
```

Q12. Optimize a multi-table JOIN with bad cardinality estimates.

```
-- Original slow query
EXPLAIN ANALYZE
SELECT o.order_id, c.name, p.title
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id
WHERE o.status = 'pending';

-- Optimization strategies:
-- 1. Ensure statistics are fresh: ANALYZE orders;
-- 2. Partial index for 'pending' orders only
CREATE INDEX idx_orders_pending ON orders (customer_id, order_id)
WHERE status = 'pending';

-- 3. Or use CTE to force materialization
WITH pending_orders AS (
    SELECT order_id, customer_id FROM orders WHERE status = 'pending'
)
SELECT ...
```

Q13. Use EXPLAIN ANALYZE output to identify seq scan on large table.

```
-- Diagnose
EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON)
SELECT * FROM events WHERE user_id = 12345 AND event_type = 'click';

-- Fix: composite index (most selective first)
CREATE INDEX idx_events_user_type ON events (user_id, event_type);
-- Or if event_type has low cardinality:
CREATE INDEX idx_events_user_time ON events (user_id, occurred_at DESC);
```

Q14. Optimize a slow GROUP BY with COUNT(DISTINCT).

```

-- Slow: COUNT(DISTINCT) on large sets
SELECT date, COUNT(DISTINCT user_id) AS dau
FROM events
GROUP BY date;

-- Alternative: HyperLogLog approximation (pg_hll extension)
-- Or: precompute in materialized view refreshed daily

-- Exact but faster: use array_agg on a smaller table
CREATE MATERIALIZED VIEW daily_active_users AS
SELECT DATE(occurred_at) AS day, COUNT(DISTINCT user_id) AS dau
FROM events GROUP BY 1
WITH DATA;
CREATE UNIQUE INDEX ON daily_active_users (day);
REFRESH MATERIALIZED VIEW CONCURRENTLY daily_active_users;

```

Q15. Optimize a search query with LIKE '%pattern%'.

```

-- Slow: no index on LIKE with leading wildcard
SELECT * FROM articles WHERE title LIKE '%machine learning%';

-- Fix 1: pg_trgm index
CREATE EXTENSION pg_trgm;
CREATE INDEX idx_articles_title_trgm ON articles USING GIN (title gin_trgm_ops);
-- Now LIKE '%pattern%' uses the index!

-- Fix 2: Full-text search (better for natural language)
CREATE INDEX idx_articles_fts ON articles USING GIN (TO_TSVECTOR('english', title));
SELECT * FROM articles WHERE TO_TSVECTOR('english', title) @@
PLAINTEXTQUERY('machine learning');

```

Q16. Partition pruning: ensure queries use partition pruning.

```

-- Partitioned table
CREATE TABLE events (
    event_id BIGSERIAL,
    occurred_at TIMESTAMPTZ NOT NULL,
    user_id BIGINT,
    event_type VARCHAR(50)
) PARTITION BY RANGE (occurred_at);

-- Good: partition pruning works
EXPLAIN SELECT * FROM events
WHERE occurred_at >= '2024-01-01' AND occurred_at < '2024-02-01';
-- Should show: Partitions: 1 of 24

-- Bad: function on partition key prevents pruning
SELECT * FROM events WHERE DATE_TRUNC('month', occurred_at) = '2024-01-01';
-- Always use direct comparisons on the partition key

```

Q17. Use CTEs for readability without sacrificing performance.

```
-- In PostgreSQL 12+, CTEs are NOT automatically materialized
-- They're inlined by default (unlike pre-12 behavior)

-- Force materialization when result is reused multiple times:
WITH expensive_calc AS MATERIALIZED (
    SELECT user_id, SUM(amount) AS total
    FROM transactions
    WHERE status = 'completed'
    GROUP BY user_id
)
SELECT ec.user_id, ec.total, u.email
FROM expensive_calc ec
JOIN users u ON ec.user_id = u.user_id
WHERE ec.total > 10000;
```

Q18. Lateral joins for per-row subqueries.

```
-- Get the latest 3 orders per customer (classic N-per-group)
SELECT c.customer_id, c.email, latest.order_id, latest.order_date
FROM customers c
CROSS JOIN LATERAL (
    SELECT order_id, order_date
    FROM orders o
    WHERE o.customer_id = c.customer_id
    ORDER BY order_date DESC
    LIMIT 3
) latest;
-- LATERAL + index on (customer_id, order_date DESC) is very efficient
```

Q19. Avoid N+1 with a single aggregation query.

```
-- N+1: app fetches each customer's last order separately – BAD
-- Fix: one query with DISTINCT ON or window function
SELECT DISTINCT ON (customer_id)
    customer_id, order_id, order_date, total_amount
FROM orders
ORDER BY customer_id, order_date DESC;
```

Q20. Use FILTER clause instead of multiple COUNTs.

```
-- Verbose, 3 passes:
SELECT
    COUNT(CASE WHEN status='completed' THEN 1 END),
    COUNT(CASE WHEN status='cancelled' THEN 1 END),
    COUNT(CASE WHEN status='pending' THEN 1 END)
FROM orders;

-- Concise, same performance:
```



```

SELECT
    COUNT(*) FILTER (WHERE status = 'completed') AS completed,
    COUNT(*) FILTER (WHERE status = 'cancelled') AS cancelled,
    COUNT(*) FILTER (WHERE status = 'pending') AS pending
FROM orders;

```

Q21. Materialized path for hierarchical queries (category trees).

```

CREATE TABLE categories (
    category_id SERIAL PRIMARY KEY,
    name VARCHAR(200),
    path LTREE -- e.g., 'Electronics.Phones.Smartphones'
);
CREATE INDEX idx_categories_path ON categories USING GIST (path);

-- Find all subcategories of 'Electronics'
SELECT * FROM categories WHERE path <@ 'Electronics';

-- Find parent categories
SELECT * FROM categories WHERE path @> 'Electronics.Phones.Smartphones';

```

Q22. Use EXCLUDE constraint for non-overlapping time ranges.

```

CREATE EXTENSION btree_gist;

CREATE TABLE room_bookings (
    booking_id SERIAL PRIMARY KEY,
    room_id INT NOT NULL,
    during TSRANGE NOT NULL,
    EXCLUDE USING GIST (room_id WITH =, during WITH &&)
);
-- This prevents double-booking at the DB level!

```

Q23. Optimize with partial indexes on boolean flags.

```

-- Index only active users (a small fraction)
CREATE INDEX idx_users_active ON users (email, created_at)
WHERE is_active = TRUE;

-- Index only unprocessed jobs
CREATE INDEX idx_jobs_pending ON jobs (created_at, priority DESC)
WHERE status = 'pending';

```

Q24. Parallel query and how to enable/tune.

```

-- Check if parallel is being used
EXPLAIN SELECT SUM(amount) FROM large_transactions;

-- Key settings
SET max_parallel_workers_per_gather = 4;

```

```
SET parallel_tuple_cost = 0.1;
SET parallel_setup_cost = 1000;
SET min_parallel_table_scan_size = '8MB';
```

```
-- Force parallel on a specific query
SET force_parallel_mode = ON;
```

Q25. Write a query to detect slow-changing dimensions (SCD Type 2).

```
-- Current state view
CREATE VIEW current_customer_scd AS
SELECT customer_id, name, email, address, segment
FROM customer_history
WHERE valid_to IS NULL; -- or valid_to = '9999-12-31'

-- Update with SCD Type 2 pattern
BEGIN;
-- Expire old record
UPDATE customer_history
SET valid_to = NOW()
WHERE customer_id = :id AND valid_to IS NULL;

-- Insert new version
INSERT INTO customer_history (customer_id, name, email, address, segment,
valid_from, valid_to)
VALUES (:id, :name, :email, :address, :segment, NOW(), NULL);
COMMIT;
```

4. Database Internals Questions

PostgreSQL / Spanner Internals

- **Q:** Explain MVCC in PostgreSQL. How does it handle reads without locks?
 - **A:** Each row version has `xmin` and `xmax` transaction IDs. A reader sees a version if `xmin` is committed and `xmax` is either not committed or future. No read locks; writers create new row versions rather than updating in place.
- **Q:** What is WAL and why does it exist?
 - **A:** Write-Ahead Log ensures durability (D in ACID). Before any data file change, the change is first written to the WAL log. On crash, PostgreSQL replays WAL to reach a consistent state.
- **Q:** How does autovacuum work and why is it critical?
 - **A:** Dead row versions (from UPDATES/DELETES) accumulate as bloat. Autovacuum reclaims space and also updates statistics used by the query planner. Without it, table bloat slows queries and eventually causes transaction ID wraparound (XID wraparound — catastrophic).
- **Q:** Explain the difference between a sequential scan and an index scan. When does PostgreSQL choose each?

- **A:** If selectivity is high (> ~5–10% of table), seq scan wins because it's sequential I/O. Index scans involve random I/O (one disk page per row for heap fetches). The planner uses cost models: `seq_page_cost` (default 1.0) vs. `random_page_cost` (default 4.0, SSDs should be ~1.1).
- **Q:** What is the difference between a bitmap index scan and a regular index scan?
 - **A:** Regular index scan fetches rows one by one (many random I/Os). Bitmap index scan builds a bitmap of matching pages first, then fetches pages in order (fewer random I/Os). Better for moderate selectivity (1–10%).
- **Q:** Explain Google Spanner's TrueTime and why it enables external consistency.
 - **A:** TrueTime provides a globally synchronized clock with bounded uncertainty (a few milliseconds). By waiting out the uncertainty interval before committing, Spanner ensures timestamps reflect true wall-clock ordering — enabling external consistency without a central coordinator.
- **Q:** What is the difference between 2PL (Two-Phase Locking) and MVCC?
 - **A:** 2PL: readers block writers, writers block readers. MVCC: readers never block writers, writers never block readers — each gets a consistent snapshot. PostgreSQL uses MVCC + SSI (Serializable Snapshot Isolation) for full serializability.

5. Database Design Tasks

Task 1: Design Google Search Index Schema

```
-- Simplified search index for URL → document → inverted index
CREATE TABLE documents (
  doc_id          BIGSERIAL PRIMARY KEY,
  url             TEXT UNIQUE NOT NULL,
  content_hash    BYTEA,
  last_crawled    TIMESTAMPTZ,
  page_rank       NUMERIC(10,8),
  language        CHAR(5)
);

CREATE TABLE terms (
  term_id         SERIAL PRIMARY KEY,
  term_text       VARCHAR(200) UNIQUE NOT NULL
);

CREATE TABLE postings (
  term_id         INT NOT NULL REFERENCES terms(term_id),
  doc_id          BIGINT NOT NULL REFERENCES documents(doc_id),
  frequency       INT NOT NULL DEFAULT 1,
  positions       INT[] NOT NULL, -- positions of term in document
  tf_idf_score    NUMERIC(10,8),
  PRIMARY KEY (term_id, doc_id)
);
```

```
CREATE INDEX idx_postings_term ON postings (term_id, tf_idf_score DESC);
```

Task 2: Design YouTube Watch History & Recommendations

```
CREATE TABLE videos (
  video_id      BIGSERIAL PRIMARY KEY,
  channel_id    BIGINT,
  title         TEXT,
  duration_s    INT,
  category_id   INT,
  tags          TEXT[],
  upload_date   DATE,
  view_count    BIGINT DEFAULT 0
);

CREATE TABLE watch_events (
  event_id      BIGSERIAL,
  user_id       BIGINT NOT NULL,
  video_id      BIGINT NOT NULL,
  watched_at    TIMESTAMPTZ NOT NULL,
  watch_duration_s INT,
  completion_pct NUMERIC(5,2),
  device_type   VARCHAR(20)
) PARTITION BY RANGE (watched_at);

-- Co-watch pairs for collaborative filtering
CREATE MATERIALIZED VIEW video_cowatch AS
SELECT
  a.video_id AS video_a,
  b.video_id AS video_b,
  COUNT(DISTINCT a.user_id) AS users_who_watched_both
FROM watch_events a
JOIN watch_events b ON a.user_id = b.user_id
                    AND a.video_id <> b.video_id
                    AND b.watched_at BETWEEN a.watched_at - INTERVAL '1 hour'
                                   AND a.watched_at + INTERVAL '1 hour'

GROUP BY a.video_id, b.video_id
HAVING COUNT(DISTINCT a.user_id) > 100
WITH DATA;
```

Task 3: Design Google Maps Location Data

```
CREATE EXTENSION postgis;

CREATE TABLE places (
  place_id      BIGSERIAL PRIMARY KEY,
  name          VARCHAR(500),
  place_type    VARCHAR(50),
```

```

    location          GEOMETRY(POINT, 4326) NOT NULL,
    address            JSONB,
    rating             NUMERIC(3,2),
    review_count       INT DEFAULT 0
);

CREATE INDEX idx_places_geo ON places USING GIST (location);
CREATE INDEX idx_places_type ON places (place_type);

-- Find restaurants within 1km of a point
SELECT name, place_type,
       ST_Distance(location::geography, ST_SetSRID(ST_MakePoint(:lon, :lat),
4326)::geography) AS dist_m
FROM places
WHERE place_type = 'restaurant'
      AND ST_DWithin(location::geography, ST_SetSRID(ST_MakePoint(:lon, :lat),
4326)::geography, 1000)
ORDER BY dist_m;

```

6. Distributed Systems + PostgreSQL Discussions

Topics Google Interviewers Probe

Consistency vs. Availability (CAP Theorem)

- PostgreSQL primary = CP (consistent + partition-tolerant)
- Aurora reader replicas = AP (may serve stale data)
- Spanner = CP with TrueTime-based external consistency

Replication Lag

```

-- Check replication lag on replica
SELECT now() - pg_last_xact_replay_timestamp() AS lag;
SELECT * FROM pg_stat_replication; -- on primary

-- Application handling: route reads that tolerate lag to replica
-- Route reads that require freshness to primary

```

Consensus and Leader Election

- Patroni + Etcd/ZooKeeper for PostgreSQL HA
- Paxos-based approaches (Spanner, CockroachDB)
- How split-brain is avoided: only one node can be leader

Sharding Strategies

- Hash sharding: even distribution, no range queries
- Range sharding: range queries work, hotspots possible
- Geographic sharding: data locality, latency benefits
- Citus extension for PostgreSQL horizontal sharding

7. Behavioral Questions

Googleness & Leadership

- "Describe a time you improved a system's reliability significantly."
- "Tell me about a technical disagreement and how you resolved it."
- "How have you worked with ambiguous requirements on a data project?"
- "Give an example of a time you went beyond your role to help the team."

Role-Specific (Data Engineering / SWE DB focus)

- "Tell me about the most complex SQL query you've ever written. Walk me through it."
 - "Describe a schema migration you managed in production. What went wrong?"
 - "How did you handle a situation where a database was causing an outage?"
-

8. Mock Interview Script

SQL Problem: "Find employees who earn more than the average salary in their department, and rank them."

Interviewer: "Let's start with a SQL problem. Given an `employees` table with `employee_id`, `name`, `department_id`, and `salary`, find all employees who earn above their department's average. Rank them within their department."

Candidate walkthrough: "I'll use a window function to avoid a subquery. Let me think about the structure..."

```
WITH dept_stats AS (  
    SELECT  
        employee_id,  
        name,  
        department_id,  
        salary,  
        AVG(salary) OVER (PARTITION BY department_id) AS dept_avg,  
        RANK() OVER (PARTITION BY department_id ORDER BY salary DESC) AS dept_rank  
    FROM employees  
)  
SELECT employee_id, name, department_id, salary, dept_avg, dept_rank  
FROM dept_stats  
WHERE salary > dept_avg  
ORDER BY department_id, dept_rank;
```

Interviewer: "What's the time complexity of this approach?"

Candidate: "The window function makes a single pass over the table, $O(n)$. The alternative using a correlated subquery would be $O(n^2)$. With an index on `(department_id, salary DESC)`, the window function can run in $O(n)$ time with efficient disk access."

Interviewer: "How would this scale to 1 billion rows?"

Candidate: "For 1B rows, I'd partition by `department_id` (or date) and run this in parallel workers. Or pre-aggregate dept averages in a materialized view and JOIN — reducing the live aggregation. Could also push this to BigQuery which handles petabytes naturally with distributed GROUP BY."

9. Evaluation Rubric

L4 (Mid-Level Software Engineer)

Signal	Meets Bar	Exceeds Bar
SQL Correctness	Solves medium problems	Handles NULL, edge cases, performance
Query Optimization	Knows indexes exist	Understands execution plans
DB Design	Normalized schema	Discusses indexing + trade-offs
Distributed Systems	Aware of replication	Can discuss consistency models
Code Quality	Correct but verbose	Clean, idiomatic SQL

L5 (Senior Software Engineer)

Signal	Meets Bar	Exceeds Bar
SQL Complexity	Solves hard problems	Optimizes and explains trade-offs
Internals	MVCC, WAL, vacuum	Deep understanding of execution
System Design	Complete designs	Discusses failure modes + SLAs
Scale	Understands partitioning	Knows Spanner/BigQuery paradigms
Leadership	Technically guides discussion	Has led DB design decisions

L6 (Staff)

Signal	Meets Bar	Exceeds Bar
Novel Solutions	Knows known techniques	Invents new approaches
Cross-System	Designs complete data stacks	Articulates strategic trade-offs
Influence	Has driven org-level changes	Can describe multi-team impact

10. Red Flags

- Cannot explain why a query is slow
- Never uses window functions — solves everything with correlated subqueries
- Ignores distributed systems entirely in design questions
- LP/behavioral stories lack specificity ("I helped improve performance" — not a story)
- Cannot distinguish EXPLAIN output between a seq scan and index scan
- Doesn't ask clarifying questions before designing
- Writes functionally correct SQL with no thought for maintainability
- Cannot discuss consistency trade-offs in replication

11. Green Flags

- Proactively mentions EXPLAIN ANALYZE before optimizing
- Discusses NULL handling without prompting
- Asks about data volume, access patterns, SLAs before designing
- Can explain the difference between WHERE and HAVING clearly
- Knows when to use CTEs vs. subqueries vs. temporary tables
- Mentions monitoring/observability in every system design
- Has hands-on experience with BigQuery or Spanner (Google-specific bonus)
- Can compare PostgreSQL and Spanner trade-offs coherently

12. Salary Bands (Approximate, USD, 2024–2025)

Level	Base	Annual Bonus	RSU (4yr)	Total Comp
L3	\$150K–\$175K	15%	\$80K–\$150K	\$185K–\$240K
L4	\$175K–\$220K	15%	\$150K–\$300K	\$240K–\$350K
L5	\$220K–\$270K	20%	\$300K–\$600K	\$360K–\$570K
L6	\$270K–\$330K	25%	\$600K–\$1.2M	\$560K–\$900K
L7	\$300K–\$375K	30%	\$1M–\$2M+	\$900K–\$1.5M

Mountain View/NYC rates. London ~70% of US. Zurich ~80%.

13. Preparation Timeline (4-Week Plan)

Week 1: SQL Mastery

- Day 1–2: Window functions (all types, all frame options)
- Day 3–4: Recursive CTEs, graph queries, hierarchical data
- Day 5: EXPLAIN ANALYZE deep dive — read 20 execution plans
- Day 6–7: LeetCode Hard SQL problems (top 30 by Google frequency)

Week 2: Database Internals

- Day 1–2: MVCC internals — read the PostgreSQL docs on row visibility
- Day 3: WAL, checkpoints, crash recovery
- Day 4: Query planner: statistics, join types (hash/nested loop/merge), cost model
- Day 5: Indexing deep dive: B-tree, GIN, GiST, BRIN, partial, composite
- Day 6–7: Distributed systems: CAP, consensus, Spanner TrueTime, CockroachDB

Week 3: System Design

- Day 1–2: Design YouTube at scale (DB focus)
- Day 3–4: Design Google Search index (inverted index, ranking)
- Day 5: Design Google Maps (PostGIS, spatial indexes)
- Day 6–7: Mock interviews (peer or recording yourself)

Week 4: Polish & Behavioral

- Day 1–2: Behavioral prep — write 8 STAR stories
- Day 3: BigQuery specifics (partitioning, clustering, query optimization)
- Day 4: Cloud Spanner vs. PostgreSQL comparison

- Day 5-6: Timed mock interviews
- Day 7: Rest, light review of cheat sheets